

Sparsity Is Not a Budget

In a trained BDH, the fraction of active neurons in layer 2 falls roughly 3x the moment the next letter becomes predictable — with the same weights, the same input length, and no sparsity setting touched anywhere.

A small instrument for a precise question

DataForge makes a sparse activation measurement inspectable. Its audience is a student or practitioner familiar with ReLU and next-token prediction. The learner should predict what repetition changes, inspect the active neurons, and find where the headline stops applying. Sparse activity is interesting because fewer nonzero coordinates may enable selective computation, but counts alone do not measure runtime, energy, or useful reasoning. Spark Transformer explicitly controls sparsity through top-k masking; that design pressure is one point of comparison, not a description of all Transformers. [3]

Mechanism and evidence

BDH's published Definition 4 gives the gated activation $y = \text{ReLU}(D_y \text{LN}(a^*))$ (elementwise product) x . Both factors are nonnegative; either zero factor closes a coordinate's gate. Our measured tensor is the product, xy_sparse , rather than the ungated projection. This structural fact does not establish why training changes either factor. The published synthetic experiment uses 65,536 neurons and four layers; its Figure 14 reports 4.0–7.5% activity during memorization and approximately 2.5% during repetition. Those paper-scale numbers are separate from our measurements. [1]

Our independent model has 1,024 measured neurons, width 32, four heads, and four applications of shared weights: 100,352 parameters. Training uses 2,200 AdamW steps on synthetic cycles: 13 fixed warm-up letters followed by eight repetitions of a random eight-letter word. The preserved evaluation word is `mmgtfhhe`. In zero-indexed layer 2, mean activity falls from 15.77% during its first appearance to 5.08% across subsequent appearances: 3.10x. The random-weight control gives approximately 1.04x. This supports training dependence in this experiment; it does not isolate a universal causal mechanism.

Comparison	BDH probe	ReLU Transformer control
Measured coordinate	Gated nonnegative product	Hidden ReLU activation
Activity denominator	1,024	1,024
Parameter count	100,352	71,680
Structural rule	Two positive factors required	Positive preactivation required

PRECOMPUTED on the canonical preset, the trained Transformer gives 1.98% versus 1.25% activity at layer 2, a 1.58x ratio, and passes the predeclared predictive-comparability gate. It also becomes sparser; the table does not establish superiority. Fixed results must not respond to sandbox controls. Equal denominators do not equalize parameter budgets, attention masks, positional dimensions, or learned function. The original Transformer equation also includes output projection and biases, omitted in the simplified hidden expression. [5]

The counterexample matters

Warm-up is predictable, yet layer-2 activity stays high. Its target-aligned model cross-entropy is 0.113 bits, below repetition's 0.766 bits. Thus a simple explanation that activity tracks model uncertainty fails here. The historical warm-up activity mean is 18.14% over 13 tokens; visible summaries exclude structurally silent token zero and give 19.65% over 12. A cold-start state-establishment explanation remains an untested hypothesis. Hidden-state prediction research independently cautions that next-token loss alone need not characterize computation; we do not implement its metric. [2]

What the learner can trust

BDH activity/CE and the separate Transformer curve and synchronized grid are LIVE; the sequence is SYNTHETIC; oracle surprisal is ANALYTIC; trained weights and offline control results are PRECOMPUTED. Grid playback is ANIMATED inspection of computed activations, not new inference per frame. Python-to-JavaScript checks compare finite outputs, exact counts, exact zero patterns, and numerical error. Layer 0 lacks the comparable drop. One cold cycle, one seed, a 64-fold neuron shrink, and synthetic training limit generalization. Platform retraining changes exact values; shipped-fixture parity is a separate check. Dense browser computation does not demonstrate sparse-kernel savings or interpretable synaptic state.

BDH-CQ is a later recurrent latent reasoning system; this artifact neither implements nor evaluates it. [4] **Not an official BDH model.** Local notes credit earlier Claude assistance; this build also uses Codex and helper agents. Editable sources, code, notices, and reproduction commands accompany the PDFs.

Sources

[1] Kosowski et al. (2025), *The Dragon Hatchling*, Definition 4; §6.4, Fig. 14.
[2] Herrmann, Csordas, Schmidhuber (2025), *Measuring In-Context Computation Complexity via Hidden State Prediction*, abstract.
[3] You et al. (2025), *Spark Transformer: Reactivating Sparsity in FFN and Attention*, abstract.
[4] Engdahl et al. (2026), *BDH-CQ: In-Context Learning with Recurrent Latent Reasoning*.
[5] Vaswani et al. (2017), *Attention Is All You Need*, equation 2.